SECURITY ASSESSMENT

BugPoCer Scan Report

FULL SCAN

REPOSITORY	FablesFi/fables
COMMIT	0f59859
BRANCH	main
SCANNED	29 October 2026

Prepared by **Olympix** · for **FablesFi**

Table of Contents

1. About

1.1. Methodology

1.2. Disclaimer

1.3. Project Overview

1.3.1. Design Goals

1.3.2. Key Patterns

1.3.3. External Dependencies

1.3.4. Invariants

1.3.5. Security Assumptions

2. Scope

3. Ingested Documentation

4. Summary

5. Findings

5.1. Medium

5.1.1. Fee-window clamp distorts stable decaying fee

5.1.2. Manipulable Spot Anchor

1. About

This report was generated by Olympix's BugPoCer tool. BugPoCer automates end-to-end vulnerability exploitation for smart contracts, moving beyond static warning to deliver concrete, verifiable Proof-of-Concept (PoC) exploits. Built on Olympix's proprietary infrastructure and Intermediate Representation (IR), BugPoCer combines results from multiple analysis engines to identify confirmed security issues. Using an agentic AI framework to synthesize insights across the Olympix toolchain, it enables teams to instantly validate vulnerabilities, measure real-world impact, and confirm that patches are effective.

1.1. Methodology

BugPoCer breaks down a codebase into source *units* (function, contract, or library level). While findings will only be searched for in units within the specified scope, BugPoCer explores all relevant paths within the codebase. It couples this context with an extensive knowledge store to identify potential issues. For each issue, BugPoCer automatically generates a ready-to-run Foundry-format PoC test that reproduces the exploit under realistic attack conditions.

EACH FINDING INCLUDES

- **Vulnerability details**: description, severity, affected lines, and potential impact
- **Exploit demonstration**: a PoC test that reliably triggers the issue

Findings are organised by the defect they represent, not by how confidently BugPoCer could demonstrate it. Several findings that share one root cause and one fix are presented together as a single numbered issue, with each affected site listed beneath it. The issue count therefore reflects the work to be done, and the finding count reflects where it has to be done.

Every finding carries a badge with **BugPoCer's assessment** of it.

Verified findings should be taken into careful consideration. BugPoCer, as any human auditor, can make mistakes, so findings it ruled out may still be worth some review time as a sanity check. They are collected in their own section when included.

BADGES ON EACH FINDING

- **Verified**: BugPoCer wrote a proof-of-concept that runs and demonstrates real harm.
- **Needs Further Review**: BugPoCer could not prove it either way; worth a manual review.

1.2. Disclaimer

This report does not guarantee the discovery of every vulnerability in scope, nor does it ensure that no new issues will arise as the codebase evolves. BugPoCer cannot provide assurances about code added or modified in future iterations. We recommend ongoing security reviews and the use of additional security measures such as independent audits and bug bounty programs.

Findings and suggested remediations in this report are provided for guidance only. Code samples or PoCs are illustrative and may require further testing and adaptation before production use.

This report is intended solely for informational purposes. It should not be interpreted as legal, financial, or investment advice, nor as an endorsement of the audited project.

1.3. Project Overview

Items marked *stated during context validation* were supplied or confirmed by the person who validated this scan's context. Everything else was inferred by BugPoCer.

NAME	Fables
TYPE	AMM / DEX Hooks (Uniswap v4 dynamic-fee hooks with LP ledger, ve(3,3))
DESCRIPTION	Fables is a set of Uniswap v4 hooks for a ve(3,3) DEX on Robinhood Chain. Each hook sets a dynamic LP fee in `beforeSwap`: either a per-direction fee inside admin bands, with keeper pokes, sender discounts and a continuation surcharge, or Uniswap's StablePair fee around a reference price. Each hook also runs an LP ledger in which deposits mint ERC-6909 receipts 1:1 with liquidity. Receipts can be staked so that their fees go to gauge voters, and a claim fee can be taken from unstaked fees.

1.3.1. Design Goals

- Hard 10% (100,000 pips) ceiling on the LP fee on every build
- LP principal stays exitable: withdraw/unstake cannot be blocked by any role, pause or configuration
- Gas-efficient swaps: assembly beforeSwap entries, packed one-word pool config, Bloom pre-filter for discounts
- Bounded keeper powers: pokes, reference moves and minimum fee are rate-limited and stay inside admin-set bounds
- Faithful StablePair math: same fee math, revert bytes and views as Uniswap's StablePair apart from documented changes
- Immutable hooks (no proxy, no upgrade), with access control through an OpenZeppelin AccessManager

1.3.2. Key Patterns

- Uniswap V4 Dynamic-Fee Hook (beforeInitialize + assembly beforeSwap fee override)
- ERC-6909 LP receipts (1:1 with liquidity units, per range id)
- OpenZeppelin AccessManaged (role-based via external AccessManager)
- Per-range LP fee ledger with Q128 reward-per-share accumulators (MasterChef style) and staked/unstaked split
- ve(3,3) gauge staking: staked receipts route fees to a distributor gated by a Voter ballot gate

1.3.3. External Dependencies

- Uniswap V4 PoolManager
- OpenZeppelin AccessManager
- WETH9

- Fables ve(3,3) Voter (IBallotGate), ve escrow and Distributor (IDistributorFables)
- Arbitrum ArbSys precompile (0x64) on Robinhood Chain

1.3.4. Invariants

High — funds/access risk Medium — functional correctness Low — informational

INFERRED BY BUGPOCER

Properties inferred by automated analysis of code patterns, architecture, and documentation.

76 INVARIANTS

On every build, the LP fee a swap pays (the fee returned with `OVERRIDE_FEE_FLAG` by the `beforeSwap` entry) must be <= the pool cap and <= 100,000 pips (10%). Fee-engine builds: the result of `FeeRef.resolve` /the assembly fallback is clamped to `w.cap()` after the keeper/poke step and again after the continuation-fee surcharge. `configurePool` requires cap <= category `capMax` <= `ABSOLUTE_MAX_FEE` (100_000). Stable builds: `_toPips` caps at `STABLE_MAX_FEE` (100_000), and `setMinimumFee` takes a `uint16`.

For the same state, the fee a swap pays must equal what the fee views quote. Fee-engine builds: the assembly `fallback` in `FablesFeeFull` must match `FeeRef.resolve` (`_resolveFeeRef`) bit for bit. `currentFee` is sender-free (no discount, CF included), and `feeFor(poolId, zeroForOne, router)` is the exact fee with the discount for `router`, the contract that calls `PoolManager.swap`. Stable builds: the assembly `fallback` around `_stableFee` must match `getFee`, `currentFee` and `feeFor(key, zeroForOne, router)`. These include the clamped price, the 10% cap, the minimum fee and the discount.

No role, pause or configuration may block `withdraw`. `withdraw` has no `whenNotPaused` / `whenPoolNotPaused` modifier. It syncs the range, settles the user, auto-unstakes any shortfall from staked shares, burns the receipts and removes liquidity through `_modify`. Fees are credited, not paid. `unstake` and `syncRanges` are also documented as never pausable. `withdrawAndClaim` is pausable (whenPoolNotPaused), so it is not the guaranteed exit. Admin-settable state that `withdraw` touches includes the claim-fee history walk, the distributor, sinks and the pause. None of it may make `withdraw` revert. Examples: a saturating `_accrue`, and the comment 'a revert would block the holder's withdraw'.

The ledger must never owe more than it holds. Sum of all holders' `owed0/owed1`, plus accrued-but-unsettled unstaked fees, plus `distributorOwed[currency]`, plus `claimFeeOwed[currency]`, must be <= the hook's PoolManager ERC-6909 claim balance for that currency. Claims are minted in `LedgerLibV2.creditFees` (`poolManager.mint`) and burned in payout/sweeps. LP principal must be backed by the hook's PoolManager positions (totalShares == position liquidity per range). Floors in `_split` and `_takeStakedCut` leave dust in the hook and never over-credit.

Fees earned before a deposit, stake or transfer cannot be taken by the newcomer.

Only the hook itself may initialize its pools. `FablesFeeEngine.beforeInitialize` reverts `InvalidInitializer` unless `sender == address(this)`, and `FablesStableFee.beforeInitialize` always reverts; v4 skips the hook on its self-call. Pools are created through `initializePool` (restricted, needs `maxFee[id] != 0`) or `configureStablePool`. On the Exclusive builds only the ledger may add liquidity: `FablesExclusiveLiquidityV2.beforeAddLiquidity` rejects outside senders, and the ledger's own adds skip the callback (noSelfCall). Removes and donates stay open.

FEE_POKER may move each side's fee within [max(floor, 50% of base), min(pool cap, category cap)] for at most 72 h, raising at most one step per 60 s; and turn the continuation fee on or off.

ADMIN must not be able to move LP principal, set a fee above 10%, or un-configure a pool. Code: `setCategory` requires `capMax <= ABSOLUTE_MAX_FEE` (100_000). `configurePool` requires 1 <= cap <= capMax and writes `maxFee[id] = cap` (never 0, so a configured pool stays configured). No admin function moves `_positions`/receipt balances or calls `take` on principal. Admin setters for sinks (`setDistributor`, `setFFFeeSink`, `setClaimFee`) redirect only fee revenue (`distributorOwed`, `claimFeeOwed`).

PAUSER may pause deposits, claims, stakes, transfers and revenue sweeps for up to 7 days per call; clear pokes; switch off the continuation fee; revoke discounts.

REVENUE_OPS may set the claim fee to at most 20% of LP fees and must not touch principal. `LedgerLibV2.recordClaimFee` reverts if bps > `MAX_CLAIM_FEE_BPS` (2000). The claim fee is applied in `_split` only to the unstaked residue of collected fees (net = fee - stakedCut). Principal (callerDelta - feesAccrued) is never charged.

On stable builds the keeper may move a pool's reference by at most 0.1% per minute and set its minimum fee (at most 65,535 pips).

Fee per swap direction, set within admin bounds; optional sender discounts and continuation fee.

LPs deposit through the hook, receive ERC-6909 receipts 1:1 with liquidity, and can stake them for gauge emissions.

Inside the optimal band both pay a small fee; outside, swaps away pay 0 and swaps back pay a falling fee.

The fee is capped at 10%; a price more than 10% from the reference is treated as exactly 10% away; the reference can move by at most 0.1% per minute.

Each hook prices swaps with a dynamic fee and holds an LP ledger with ERC-6909 receipts 1:1 with liquidity.

`FablesFeeEngine._word` base0/base1/floor/cap and `FablesHookCore.maxFee`: set by ADMIN via `FablesFeeEngine.configurePool`, bounded to `category.floorMin <= floor <= min(base0, base1)`, `max(base0, base1) <= cap <= category.capMax`, `cap >= 1`, key.hooks == this, dynamic-fee key; `maxFee[id] = cap` (never returns to 0); pokes, bloom/seed and CF are kept. Read by the beforeSwap `FablesFeeFull` fallback and `FablesFeeFull._resolveFeeRef` via `FeeRef.keeperFee` (fee = base(dir), or with a live poke max(poke, max(floor, min(base,cap)>>1)), then min(fee, cap)) and `FeeRef.resolve` (CF surcharge then capped at cap); by `FablesFeeFull.currentFee` / `feeFor`; and `maxFee` by `FablesLedgerV2._checkKey` and `FablesFeeEngine.initializePool`.

`FablesFeeEngine._word` poke fields (fee0For1, fee1For0, expiry) and `FablesFeeEngine._meta.lastRaiseAt`: set by FEE_POKER via `FablesFeeEngine.pokeFee`, bounded to each non-zero side in [floor, min(cap, category.capMax)], `1 <= ttl <= MAX_POKE_TTL` (72h), expiry = now + ttl; if either side's resolved `FeeRef.keeperFee` rises, the rise must be <= category.raiseStep and `now >= lastRaiseAt + RAISE_INTERVAL` (60s), then lastRaiseAt = now; cuts are unbounded down to the resolve-time floor max(floor, min(base,cap)/2). Cleared by PAUSER via `FablesFeeEngine.clearPoke` (no bound). Read by `FeeRef.keeperFee` in the beforeSwap fallback, `FablesFeeFull.currentFee`, `FablesFeeFull.feeFor`, `FablesFeeEngine.pokeOf`.

`FablesFeeEngine._entry[poolId][account]` (bps, minFee, dirs, expiry): set by DISCOUNT_ADMIN via `FablesFeeEngine.setDiscount`, bounded to account != 0, `bps <= MAX_DISCOUNT_BPS` (5_000), and when bps > 0: dirs in 1..3, `block.timestamp < expiry <= block.timestamp + MAX_DISCOUNT_TERM` (90 days); `minFee` is an unbounded uint24; pool must have cap != 0; bps == 0 deletes; also ORs the account's bit into the pool bloom. Deleted by PAUSER via `FablesFeeEngine.revokeDiscount`. Read on a bloom hit by `FeeRef.resolve` -> `DiscountLib.applyEntry` / `applyDiscount`: fee' = min(fee, max(ceil(fee* (10_000 - min(bps,5_000))/10_000), pool floor, minFee)) when live for the direction and now < expiry; applied before the CF surcharge. Readers: beforeSwap fallback, `FablesFeeFull._resolveFeeRef`, `FablesFeeFull.feeFor`.

`FablesFeeEngine._cf` (mode, kQ8, cfCap, band16, tau, period, maxAge, anchor, tLast): set by ADMIN (unbound selector default) via `FablesFeeEngine.configureCF`, bounded to mode <= 1, 0 < kQ8 <= MAX_CF_K_Q8 (256), 0 < cfCap <= MAX_CF_CAP (2_500 pips), band16 <= MAX_CF_BAND16 (800), MIN_CF_TAU (5) <= tau <= MAX_CF_TAU (300), 1 <= period <= MAX_CF_PERIOD (60), period < maxAge = min(255, 5*tau); reverts `CFActive` while cfOn; anchor = current slot0 tick * 256, tLast = now. Re-anchored by FEE_POKER via `FablesFeeEngine.setCFActive(true)` (current tick), kQ8 zeroed by PAUSER via `FablesFeeEngine.killCF`. Read in the beforeSwap fallback / `FablesFeeFull._resolveFeeRef` by `FeeRef.cfStep`: if now - tLast >= period, A = T + (A - T)decay$^{dt / 2}$32 with T = tick256 and the word is rewritten; x = (zeroForOne ? A - T : T - A) (negated in reverse mode) - band16*16; surcharge s = min((kQ8 * x * 100) >> 16, cfCap) when x > 0; `FeeRef.resolve` adds s to the fee and clamps to the pool cap.

LP principal entry: `FablesLedgerV2.deposit` (nonReentrant, `whenPoolNotPaused`, deadline, `_checkKey`) registers `_rangeKeys[id]` on first use, calls `_syncRange(id)` then `_settleUser(id, msg.sender)`, bumps `resetSeq` if `totalShares == 0`, raises `_ranges[id].totalShares += liquidity`, `_mint(msg.sender, id, liquidity)`, then `_modify(+liquidity)` → unlock → `LedgerLibV2._onModify` → `poolManager.modifyLiquidity(salt 0)`; `_settlePrincipal` computes `principal = callerDelta - feesAccrued` per currency, checks paid ≤ `amount0Max` / `amount1Max`, and `_resolvePrincipal` pulls the negative amount with `IERC20.safeTransferFrom(user, poolManager, owed)` + `settle()` (or `settle{value: owed}` for native currency0, where `msg.value >= amount0Max` is required and `_refundNative` returns `address(this).balance - foreignNativePre` to `msg.sender` last).

withdraw has no pause modifier and is not gated by a role; whether configuration or other state can block it is not shown by the quote.

Fee collection: `_syncRange(id)` (no-op except rolling `claimFeeSyncIndex[id]` when the range is unset or `totalShares == 0`) opens a MODIFY unlock with `liquidityDelta = 0`; `_onModify` passes v4's `feesAccrued` into `LedgerLibV2.creditFees`, which prices the backlog with `claimFeeFor(history, claimFeeSyncIndex[id], ...)`, sets `claimFeeSyncIndex[id] = history.length`, mints `fee0`/`fee1` as hook-owned PoolManager ERC-6909 claims (`poolManager.mint(address(this), currency.toId(), fee)`), and `_split`s them. `_onModify` always credits at the totals in storage at the time of the modify (post-burn for withdraw, post-mint for deposit).

Fee split in `LedgerLibV2._split`: with `total = range.totalShares`: if `total == 0` all `fee0/fee1` go to `distributorOwed`; else staked cut `cut = floor(fee * totalStaked / total) + (carried >> 128)` (Q128 remainder carried in `stakedResidual0/1`) is added to `distributorOwed` and `accStaked*X128 += cut * 2^128 / totalStaked`; if `unstakedTotal = total - totalStaked == 0` the rest also goes to `distributorOwed`; otherwise net = fee - cut, claim fee `c = net * feeBps / 10_000` (only when `feeBps != 0` and recipient set) goes to `claimFeeOwed`, and `accFee*X128 += (net - c) * 2^128 / unstakedTotal` in an unchecked block (wraps mod 2^256). Floor dust stays as hook-held claims.

settleUser adds unstaked*(accFee-checkpoint)/2^128 to owed, saturating at MAX_OWED.

`FablesStableFee._words.cfg` k / optimalFeeE6 / targetMultiplier: set by ADMIN via `FablesStableFee.configureStablePool` (once per pool; `_referenceOf(poolId) != 0` reverts `PoolAlreadyConfigured`) and `FablesStableFee.updateFeeConfig`, bounded by `_validateFeeConfig` to `k != 0` (no upper bound), `optimalFeeE6 <= MAX_OPTIMAL_FEE_E6` (1e4 = 1%), `1 <= targetMultiplier <= MAX_TARGET_MULTIPLIER` (100); both reset the fee state (`_resetFeeState`: UNDEFINED decaying fee, current clock, cached price zeroed); whitelist bits [216,256) are kept; `configureStablePool` writes `maxFee[id] = STABLE_MAX_FEE`. Read by `FablesStableFee._getFee` / `_calculateDecayingFee` through `_stableFee` in `FablesStableHookLite` fallback and `FablesStableHookLiteTyped.beforeSwap`, and by views `getFee`, `currentFee`, `feeFor`, `feeConfig`; the charged fee is `_toPips(lpFeeE12)` capped at STABLE_MAX_FEE (100_000).

`FablesStableFee._words.cfg` referenceSqrtPriceX96 and `FablesStableHookLite.referenceUpdatedAt`: first set by ADMIN via `FablesStableFee.configureStablePool` (validated only by `_validateReferenceSqrtPriceX96`: the band [ref*sqrt(1-1%), ref/sqrt(1-1%)] fits v4's sqrt price range; write-once); moved by STABLE_KEEPER via `FablesStableHookLite.updateReferencePrice`, bounded to `block.timestamp >= referenceUpdatedAt + REFERENCE_UPDATE_INTERVAL` (60s), `_validateReferenceSqrtPriceX96`, and `calculatePriceRatioX96(new, previous) >= MIN_REFERENCE_RATIO_X96` (= Q96(1e6-1_000)/1e6, i.e. a price step <= 0.1% either direction per call); resets the fee state and sets referenceUpdatedAt = now. `updateFeeConfig` keeps the reference. Read by `FablesStableFee._getFee` (price ratio to reference; ratios < MIN_FEE_PRICE_RATIO_X96 (0.9) are clamped to the window edge; side ammPriceBelowRP picks which direction pays the decaying fee), `_checkPoolInitialized` (reference != 0 marks an initialized pool), `feeConfig`.

`FablesStableFee._words.state` minimumFee bits [240,256): set by STABLE_KEEPER via `FablesStableHookLite.setMinimumFee`, bounded only by its uint16 type (0..65_535 pips); lowering it calls `_resetFeeState`. Read by `FablesStableFee._getFee`: lpFeeE12 = max(lpFeeE12, minimumFee * 1e6) on every swap (both directions, inside and outside the band), returned in stateLow; by `FablesStableFee._discounted` as the discount floor (`DiscountLib.applyEntry(fee, minimumFee, entry, ...)`); by `FablesStableHookLite.minimumFee`. `_resetFeeState` and the per-block state write preserve these bits.

`FablesStableFee._entry[poolId][account]` and cfg bloom32/seed8 bits [216,256): entries set by DISCOUNT_ADMIN via `FablesStableFee.setDiscount` (account != 0, bps <= MAX_DISCOUNT_BPS (5_000); when bps > 0: dirs 1..3, now < expiry <= now + 90 days; minFee unbounded uint24; pool must pass `_checkPoolInitialized`; ORs `DiscountLib.mask32` bit into bloom), filter rebuilt via `FablesStableFee.setDiscountFilter` (seed any uint8, bloom from listed accounts holding an entry), entries deleted by PAUSER via `FablesStableFee.revokeDiscount` (no pool check). Read by `FablesStableFee._discounted` from `_stableFee` (only when cfg>>216 != 0, fee != 0, and hit32): fee' = min(fee, max(ceil(fee*(1-min(bps,50%))), minimumFee, entry.minFee)) for a live direction before expiry; the discount floor here is the pool minimumFee, not a pool floor.

FablesHook builds clamp keeperFee and CF sum to the pool cap, and stable builds cap _toPips at 100,000.

`FablesLedgerV2._poolClaimFee` (bps, history, allTimeMin): set by ADMIN via `FablesLedgerV2.setClaimFee` and by the role bound to `FablesLedgerV2.setClaimFeeBps` (REVENUE_OPS in the fixture; reverts while claimFeeRecipient == 0), bounded by `LedgerLibV2.recordClaimFee` to `bps <= MAX_CLAIM_FEE_BPS` (2_000), bps != 0 needs a non-zero recipient; each call pushes {fromTs: now, bps: recipient==0 ? 0 : bps} and updates allTimeMin. Read by `LedgerLibV2.claimFeeFor`: rate = min over history entries since the range's `claimFeeSyncIndex` (walk <= MAX_CLAIM_FEE_WALK, else min with allTimeMin), capped by the current bps, 0 if sinceIndex == 0 or no recipient; by `LedgerLibV2.creditFees` -> `_split`: claim cut c = net * feeBps / 10_000 on the unstaked residue only, credited to `claimFeeOwed`; by `FablesLedgerV2._effectiveClaimFee` (0 when fully staked) checked against the user's `maxFeeBps` in `claimFees` and `withdrawAndClaim`; by `effectiveClaimFee`, `claimFeeBps`, `claimFeeAllTimeMin`.

`FablesLedgerV2.claimFeeRecipient` (hook-wide): set by ADMIN via `FablesLedgerV2.setClaimFee` (any pool's call rewrites it for all pools), bounded to: cannot return to zero once non-zero, != PoolManager (`_checkSinkTarget`), != this, != distributor, != ffFeeSink (`recordClaimFee`); settles `settleFirst` currencies to the old sinks first (`_settleSinks`); no allowlist otherwise. Read by `LedgerLibV2.sweepClaimFee` / `FablesNativeETHV2._sweepClaimFee` (payout target of `claimFeeOwed`; native paid as WETH), by `creditFees` / `_split` and `claimFeeFor` (zero means no claim fee), by `unlockCallback` (ClaimFeeState), and by `setDistributor` / `setFFFeeSink` collision checks.

`FablesLedgerV2.distributor` and `FablesLedgerV2.distributorFF`: set by ADMIN via `FablesLedgerV2.setDistributor` after `_settleSinks(settleFirst)`; bounded by `_checkSinkTarget` (!= PoolManager) and `LedgerLibV2.validateDistributor` (zero clears both; non-zero must != ffFeeSink, != claimFeeRecipient, != this, have code, and answer `ff()` with a non-zero token cached as distributorFF). Read by `LedgerLibV2.sweepDistributor` via `FablesLedgerV2._sweepDistributor` / `sweepDistributor` (pays `distributorOwed[currency]` to ffFeeSink when currency == distributorFF, else to distributor; waits while target is zero) and `FablesNativeETHV2._sweepDistributor` (native wrapped to WETH and sent to distributor).

AccessManager configuration and `AccessManaged` authority of every hook: role grants and selector-to-role bindings are set by ADMIN in the AccessManager (unbound selectors default to ADMIN_ROLE); the hook's authority can be replaced via `AccessManaged.setAuthority` called by the current authority. Read by the `restricted` modifier on every FablesFeeEngine / FablesStableFee / FablesStableHookLite / FablesLedgerV2 setter. Hooks have no proxy or upgrade path; `poolManager`, `weth`, `USE_ARB_SYS` are immutable constructor values.

README fee ceiling: the LP fee charged on a swap stays at or below the pool cap (`maxFee[id]` / `_word` cap) and at or below 100_000 pips on every build. Mechanisms: Full builds clamp in `FeeRef.keeperFee` and again after the CF surcharge in `FeeRef.resolve`, with cap <= category.capMax <= ABSOLUTE_MAX_FEE; stable builds clamp in `FablesStableFee._toPips` to STABLE_MAX_FEE.

Shared state `_ranges[id].totalShares`: raised only by `deposit` (`+= liquidity`, after `_syncRange` + `_settleUser`), lowered only by `_withdrawTo` (`-= liquidity`, after sync/settle and `_burn`); each transition to/from 0 bumps `resetSeq` (deposit when it was 0, withdraw when it reaches 0). A recreated range keeps its old `accFee*X128` / `accStaked*X128` levels; the depositor's checkpoint is set to them by `_settleUser`. When `totalShares == 0`, `_syncRange` does not touch the v4 position and only rolls `claimFeeSyncIndex`; any fees that `_split` sees with `total == 0` go to `distributorOwed`.

Shared state `_positions[id][owner].staked` and `_ranges[id].totalStaked`: raised by `stake` (requires `ballotGate != 0`, `isGaugeEarning(poolId)`, optional `gaugeWeightNow >= minGaugeWeight`, unstaked balance ≥ shares, pausable); lowered by `unstake` (never pausable, no gate check) and by `_withdrawTo`'s auto-unstake when `liquidity > balance - staked`. Each path calls `_syncRange` then `_settleUser` first. `_checkTransferable` blocks transfers of staked receipts (`balanceOf - staked`). `_effectiveClaimFee` returns 0 when `totalStaked == totalShares`.

`_ranges[ID].totalShares` equals the sum of `balanceOf` over all accounts; `join` and `leave` change both by the same amount.

`_ranges[ID].totalStaked` equals the sum of `_pos[who].staked` over all accounts, and `totalStaked` is at most `totalShares`. Each account's staked amount is at most its `balanceOf`. `unstake` does not check the per-account staked amount, so it relies on checked arithmetic.

A DISCOUNT_ADMIN discount must never raise a fee and never go below the floor (fee-engine builds: the pool `floor`; stable builds: the pool's minimum fee). `DiscountLib.applyDiscount` returns min(fee, max(ceil(fee*(10000 - min(bps, 5000))/10000), floor, minFee)). `setDiscount` requires bps <= `MAX_DISCOUNT_BPS` (5000), dirs in 1..3 and now < expiry <= now + `MAX_DISCOUNT_TERM` (90 days). On fee-engine builds the CF surcharge is added after the discount, then clamped to cap.

SEEDER may initialize a pool the admin has configured (FablesHook builds), at the price it passes.

CF bounds: slope <= 1.0 (Q8), surcharge <= 25 bp, dead band <= 50 bp, tau 5..300 s.

Documented choice: the claim fee on a range's backlog is charged at the LOWEST rate in force since that range's last sync. A cut applies to the whole backlog; a raise only to later backlogs. Code (`LedgerLibV2.claimFeeFor`):

- 0 if sinceIndex == 0.
- Otherwise rate = (recipient == 0 ? 0 : current bps), then the min over history entries [sinceIndex-1 .. len-1].
- The walk covers at most `MAX_CLAIM_FEE_WALK` (32) entries. If that does not cover the window, it returns min(rate, allTimeMin). `creditFees` prices the backlog with the old `claimFeeSyncIndex[id]`, then moves the index to the head. Anyone can trigger a sync through deposit, transfer or claim, and the code comment says a backlog can be priced just before a public cut.

Stable builds: every swap must pay at least the pool's minimum fee (`setMinimumFee`, default 0 = StablePair's fee), and no discount may go below it. Lowering it must restart the falling fee. Code: `_getFee` raises lpFeeE12 to minimumFee*1e6 when minimumFee != 0. `_discounted` passes the minimum fee as the floor of `DiscountLib.applyEntry`. `setMinimumFee` calls `_resetFeeState` only when the new value < the old one, and emits `MinimumFeeSet`. Off-chain models are expected to treat a lowering `MinimumFeeSet` as a fee-state reset, like `FeeConfigUpdated`.

`FablesFeeEngine._categories` (floorMin, capMax, raiseStep): set by ADMIN via `FablesFeeEngine.setCategory`, bounded to `capMax <= ABSOLUTE_MAX_FEE` (100_000 pips), `floorMin <= capMax`, `raiseStep != 0`; no bound relative to pools already in the category (they keep their stored floor/cap until reconfigured). Read by `FablesFeeEngine.configurePool` (floor >= floorMin, cap <= capMax) and `FablesFeeEngine.pokeFee` (poke ceiling = min(pool cap, capMax); `_raises` reverts if a resolved-fee rise exceeds raiseStep).

`FablesFeeEngine._word` bloom and seed fields: set by DISCOUNT_ADMIN via `FablesFeeEngine.setDiscountFilter` (bounded to `seed <= PoolWordLib.MASK7`; bloom rebuilt only from the listed accounts that hold a non-zero entry) and OR-ed by `FablesFeeEngine.setDiscount`; `revokeDiscount` leaves bloom bits. Read by `FeeRef.resolve` / `FablesFeeFull._resolveFeeRef` and the beforeSwap fallback: the `_entry` lookup and discount run only when bloom != 0 and `DiscountLib.hit64(bloom, sender, seed)`.

`FablesFeeEngine._word` cfOn flag: set by FEE_POKER via `FablesFeeEngine.setCFActive` (on requires `FeeRef.cfK(_cf) != 0` and an initialized pool, and re-anchors via `FeeRef.cfRearm(cf, now, tick)`; off has no precondition beyond a configured pool) and cleared by PAUSER via `FablesFeeEngine.killCF`. Read by `FeeRef.resolve` / `FablesFeeFull._resolveFeeRef` (CF surcharge and CF-word write only when on) and by `FablesFeeEngine.configureCF` (reverts while on).

Initial pool price: on FablesHook builds set by SEEDER via `FablesFeeEngine.initializePool(key, sqrtPriceX96)` (requires `maxFee[id] != 0` and native currency0 only on native builds; no bound on sqrtPriceX96 in the hook); on stable builds set by ADMIN via `FablesStableFee.configureStablePool(key, sqrtPriceX96, config)` (no bound on sqrtPriceX96 relative to `config.referenceSqrtPriceX96`). `FablesFeeEngine.beforeInitialize` and `FablesStableFee.beforeInitialize` refuse every initializer other than the hook itself. The price is read by v4 swaps/liquidity math, by CF anchoring (`configureCF` , `setCFActive`) and by `FablesStableFee._stableFee` / `_getFee`.

Revenue sweeps: `sweepDistributor(currency)` (anyone, nonReentrant, `whenNotPaused` hook-wide only) pays `distributorOwed[currency]` to `ffFeeSink` when `distributorFF != 0 && currency == distributorFF`, else to `distributor`; it is a no-op (accrual kept) while the target is `address(0)`, otherwise zeroes the bucket and runs `payout(currency, currency, amount, 0, to)`. `sweepClaimFee(currency)` pays `claimFeeOwed[currency]` to `claimFeeRecipient` the same way. On native builds (`FablesNativeETHV2`) a native-currency sweep pays the hook itself, `weth.deposit{value: amount}()` and `safeTransfer`s WETH to the sink (native distributor sweep ignores `ffFeeSink`). `_settleSinks` (called by `setClaimFee` , `setDistributor` , `setFFFeeSink`) sweeps the listed currencies before the sink changes.

pokeFee limits each side to [floor, min(pool cap, category cap)] for up to 72h with raise-step limits per 60s.

`FablesLedgerV2.ffFeeSink` : set by ADMIN via `FablesLedgerV2.setFFFeeSink` after `_settleSinks(settleFirst)` , bounded to (if non-zero) != distributor, != claimFeeRecipient, != this, and != PoolManager; zero allowed. Read by `LedgerLibV2.sweepDistributor` (target for distributorOwed in the distributorFF token; zero makes it wait), `LedgerLibV2.recordClaimFee` and `validateDistributor` collision checks.

withdraw does not read pausedUntil and has no pause modifier.

`FablesLedgerV2.ballotGate` : set by ADMIN via `FablesLedgerV2.setBallotGate` ; zero allowed (closes stake and syncRanges); non-zero must have code, answer `ballotBacks` , `isGaugeEarning` , `gaugeWeightNow` without reverting, and its `ve()` escrow (non-zero, with code) must return the gate from `voter()` . Read by `FablesLedgerV2.stake` (reverts unless `isGaugeEarning(poolId)` ; optional `gaugeWeightNow >= minGaugeWeight`) and `FablesLedgerV2.syncRanges` (each staked range requires `ballotBacks(msg.sender, tokenId, poolId)`).

Claim-fee rate `LedgerLibV2.claimFeeFor(history, sinceIndex, recipient, bps, allTimeMin)`: returns 0 if `sinceIndex == 0`; otherwise starts at the current `bps` (0 if no recipient) and takes the minimum `bps` over history entries from index `length-1` down to `sinceIndex-1`, walking at most `MAX_CLAIM_FEE_WALK = 32` entries; if the walk did not reach `sinceIndex-1` it returns `min(rate, allTimeMin)`. `sinceIndex` = `claimFeeSyncIndex[id]`, set to `history.length` on every `creditFees` and idle `_syncRange`. The rate is charged as `net * feeBps / 10_000` on the unstaked residue only. Inputs: `bps` by REVENUE_OPS via `setClaimFee` / `setClaimFeeBps` (`<= MAX_CLAIM_FEE_BPS = 2000`, each call appends a history entry); `sinceIndex` moves with any sync (any LP action or a voter's `syncRanges`).

The reference price can move by at most 0.1% per update, and updates are limited to once per 60 seconds.

Entry points that force `_syncRange(id)` (a v4 fee collection that raises `accFee*X128` / `accStaked*X128`, `distributorOwed`, `claimFeeOwed` and rolls `claimFeeSyncIndex`): `deposit`, `withdraw`, `withdrawAndClaim`, `claimFees` (caller must hold receipts or owed), `stake`, `unstake` (shares > 0), `transfer` / `transferFrom` (amount > 0, unstaked), and `syncRanges(tokenId, ids)` (only ranges with `totalStaked > 0` and only when `IBallotGate.ballotBacks(msg.sender, tokenId, poolId)`; never pausable). Anyone can donate to the pool via PoolManager to raise the fee growth these syncs collect.

Shared state `_positions[id][owner].owed0/owed1`: raised by `LedgerLibV2.settleUser` (called for `msg.sender` in deposit/withdraw/claimFees/stake/unstake and for both sender and receiver in `transfer` / `transferFrom`, always before the balance change); zeroed only by `claimFees` and `withdrawAndClaim` immediately before `_payout`. A holder with zero balance but non-zero owed can still call `claimFees`.

Shared state `distributorOwed[currency]` / `claimFeeOwed[currency]`: raised by `LedgerLibV2._split` during any fee collection (staked cut, `total == 0` fees, `unstakedTotal == 0` residue → `distributorOwed`; claim fee → `claimFeeOwed`); reset to 0 by `sweepDistributor` / `sweepClaimFee` (anyone, hook-wide pause gated) and by `_settleSinks` inside `setClaimFee`, `setDistributor`, `setFFFeeSink` (restricted, not pause gated). Both are keyed by currency across all pools of the hook.

Shared state `_word[poolId]` (FablesHook builds; read on every swap): bases/floor/cap written by `configurePool` (ADMIN, also sets `maxFee[id] = cap`, `_meta.category`); pokes+expiry written by `pokeFee` and cleared by `clearPoke`; bloom OR-ed by `setDiscount` (never cleared there) and replaced (with seed) by `setDiscountFilter`; `cfOn` toggled by `setCFActive` and cleared by `killCF`. `_cf[poolId]` written by `configureCF` (only while off; anchors at current tick), `setCFActive(true)` (re-arm at current tick), `killCF` (k := 0) and by the swap `fallback` (tLast/anchor). `_entry[poolId][account]` written by `setDiscount` (bps>0 pack, bps==0 delete) and `revokeDiscount`.

Shared state `_words[poolId].state` (stable builds): written by the swap `fallback` through `_stableFee` on the first swap of each clock block (decaying fee bits [0,40), cached sqrt price [40,200), clock [200,240), minFee bits preserved); reset by `_resetFeeState` (UNDEFINED fee, current clock, price 0) from `configureStablePool`, `updateFeeConfig`, `updateReferencePrice` and `setMinimumFee` when lowering; `setMinimumFee` writes bits [240,256). `_words[poolId].cfg` written by `_writeFeeConfig` (bits [0,216): k, optimalFeeE6, targetMultiplier, reference) and by the discount setters (bloom32/seed8 bits [216,256)).

Only the hook can initialize its pools; on Exclusive builds only the ledger can add liquidity.

Admins can pause deposits, claims, stakes, transfers and revenue sweeps for up to 7 days per call.

Principal slippage bound `LedgerLibV2._checkBound(principal, liquidityDelta, limit)` with `principal = callerDelta - feesAccrued`: for an add (`liquidityDelta > 0`) `paid = principal < 0 ? -principal : 0` must be ≤ `amountXMax` (caller input); for a remove `received = principal > 0 ? principal : 0` must be ≥ `amountXMin` (caller input); a 0-delta sync passes `type(uint128).max` limits. Both deposit and withdraw also check `block.timestamp <= deadline` (caller input). The pool price that sets the principal split moves with any swap in the same block.

Accumulator levels `accFee0X128/accFee1X128` and `accStaked0X128/accStaked1X128` are increased in `unchecked` blocks and wrap mod 2^256; `settleUser` reads only `_since(level, checkpoint)` differences. Their increments are `net * 2^128 / unstakedTotal` and `cut * 2^128 / totalStaked`, so a small share count combined with a large fee (e.g., a `PoolManager.donate` into the range's ticks by any user) produces large per-share increments; `owed` saturates at `int128.max`.

The fee views (`FablesFeeFull.currentFee` / `feeFor` via `_resolveFeeRef`; `FablesStableFee.getFee` / `currentFee` / `feeFor` via `_viewState` / `_getFee`) are the Solidity twins of the assembly `fallback` beforeSwap entries; the fee a swap pays equals what the views quote for the same state and sender.

Native-ETH deposit accounting: `deposit` on a native currency0 pool requires `msg.value >= amount0Max`, stores `address(this).balance - msg.value` in transient `_FOREIGN_NATIVE_SLOT`, settles the owed currency0 principal with `poolManager.settle{value: owed}()`, and `_refundNative` sends `address(this).balance - pre` to `msg.sender` (reverting on failure) as the last step; `receive()` only accepts ETH from `poolManager` (used by native sweeps, which then wrap it with `weth.deposit`). ERC20 pools require `msg.value == 0`.

setClaimFee rejects bps > 2000, i.e. at most 20% of net unstaked LP fees.

Swap discounts are looked up by the `sender` argument of `beforeSwap` (the contract calling `PoolManager.swap`, i.e., the router), not by the end trader: `FablesFeeFull._resolveFeeRef` reads `_entry[poolId][sender]` on a 64-bit bloom hit and `FablesStableFee._discounted` on a 32-bit bloom hit; any user routing through a listed router address receives that router's discount (bps ≤ 5_000, floor = pool `floor` or `minimumFee`, combined with entry `minFee`).

> `ConservationHandler.callCount` and `ConservationHandler.slackGrowths` are cumulative counters that only increase (by `++`) and never decrease.

> `OzBaseHandler.calls` per-key counters are cumulative and only move upward: each handler function (`claim`, `deposit`, `directAdd`, `directRemove`, `donate`, `ledgerRoundTrip`, `nextBlock`, `poke`, `swap`, `swapRoundTrip`, `transferReceipts`, `withdraw`) only increments its own key by exactly 1 per invocation, and no path decrements or resets it.

> `LedgerHandler.callCount` is a cumulative counter that only increases, by exactly 1 per handler call (`claim`, `deposit`, `pauseFor`, `setClaimFee`, `stake`, `stakeAll`, `swapAround`, `sweep`, `transfer`, `unstake`, `warp`, `withdraw`, `withdrawAll`, `withdrawAndClaim`). No function decreases or resets it.

> `refused` and `landed` per-key counters only increase (never decrease or reset) across `_outcome`, `_swap`, `claim`.

> `stepsDone` only increases, except when reset by `setBudget`; each handler action increments it by exactly one.

> For each key, `landed` and `refused` are cumulative counters that only increase. Each attempted outcome increments exactly one of the two, so the sum of both over all keys is at most the number of handler actions.

> `pausesLanded` is monotonically non-decreasing and increments by exactly one for each successful `pausePool` or `liftPool` that changes `expectedPausedUntil`; the shadow `expectedPausedUntil[id]` must equal the hook's pausedUntil for that pool, and `pausePool` only raises it (never shortens an active pause).

> `fieldViolations` is a cumulative counter that only increases (++ only) and must remain 0 in a correct system: any increment indicates a non-pokes field of the pool word changed unexpectedly.

1.3.5. Security Assumptions

19 ENTITIES

> **Uniswap V4 PoolManager** TRUSTED
>
> - Invokes hook callbacks honestly per IHooks permission flags
> - Enforces single-active-unlocker semantics
> - Reverts on non-zero net BalanceDelta at unlock exit
> - Does not adversarially re-enter hooks

ADMIN (AccessManager ADMIN_ROLE, the authority behind every `restricted` function) TRUSTED

- Grants/revokes every role and rebinds any restricted selector to any role (AccessManager.setTargetFunctionRole); README: 'Role-to-function bindings are AccessManager configuration set at deployment; ADMIN can change them.' Unbound selectors default to ADMIN_ROLE, so ADMIN holds every restricted function not explicitly bound
- Creates/updates fee categories via FablesFeeEngine.setCategory (capMax <= ABSOLUTE_MAX_FEE=100_000, floorMin <= capMax, raiseStep != 0)
- Configures pools via FablesFeeEngine.configurePool (floorMin <= floor <= min(base0,base1), max(bases) <= cap <= capMax, cap >= 1) and writes maxFee[id]=cap; reconfiguration keeps pokes, filter and CF state
- Configures continuation fee via FablesFeeEngine.configureCF (bounded by MAX_CF_* constants, reverts while CF is on)
- Creates stable pools via FablesStableFee.configureStablePool (write-once reference, k != 0, optimalFeeE6 <= 1e4, targetMultiplier 1..100, initial sqrtPriceX96 unbounded) and updates k/optimalFeeE6/targetMultiplier via updateFeeConfig
- Sets per-pool claim-fee rate (<= 2_000 bps) and the hook-wide claimFeeRecipient via FablesLedgerV2.setClaimFee; recipient cannot return to zero once set
- Sets distributor (contract answering ff()) via setDistributor, ffFeeSink via setFFFeeSink, ballotGate via setBallotGate; these addresses receive swept staked-share / FABLES revenue; accepts any address passing the probe checks (no allowlist)
- Lifts or sets hook-wide / per-pool pause (0..7 days) via setPaused / setPoolPaused; pause never blocks withdraw, unstake or syncRanges
- Can change the hook's authority via AccessManager.updateAuthority -> AccessManaged.setAuthority
- README 'Cannot': move LP principal, set a fee above 10%, un-configure a pool
- Hooks are immutable: no proxy and no upgrade path

REVENUE_OPS / POKER_ADMIN / PAUSER_ADMIN (deployment-configured AccessManager roles) TRUSTED STATED IN VALIDATION

- REVENUE_OPS (fixture binding) calls FablesLedgerV2.setClaimFeeBps: sets a pool's claim-fee rate (<= MAX_CLAIM_FEE_BPS = 2_000 bps) against the existing claimFeeRecipient (reverts while recipient is zero); never sweeps
- POKER_ADMIN grants/revokes FEE_POKER; PAUSER_ADMIN grants/revokes PAUSER
- Bindings are deployment configuration not present in src/; ADMIN can rebind

External sinks: distributor, ballot gate (Voter/IBallotGate) and ffFeeSink / claimFeeRecipient TRUSTED

- distributor receives staked-share fees (distributorOwed) through sweepDistributor; must answer ff() with a non-zero FABLES token (cached as distributorFF)
- ffFeeSink receives distributorOwed denominated in distributorFF
- claimFeeRecipient receives claimFeeOwed through sweepClaimFee; native revenue is wrapped and sent as WETH on ETH builds
- ballotGate answers isGaugeEarning/gaugeWeightNow (gates stake) and ballotBacks (gates syncRanges); a reverting or lying gate blocks or opens staking and voter syncs

WETH9, ArbSys precompile and pool ERC-20 tokens TRUSTED

- FablesNativeETHV2 wraps native sweeps via weth.deposit and safeTransfer to the sink
- FablesStableFee reads the block clock from ArbSys(0x64).arbBlockNumber() when USE_ARB_SYS, else block.number
- README: fee-on-transfer, rebasing and >2^127-supply tokens are unsupported; a token issuer pausing a token blocks exits that pay that token

Protocol-fee controller (Uniswap v4 on Robinhood Chain) TRUSTED

- May add up to 0.1% per side on top of the LP fee
- The total fee a trader pays can therefore exceed the hook's LP fee by up to the protocol fee

OpenZeppelin (AccessManager, ReentrancyGuardTransient) and Robinhood Chain TRUSTED

- OpenZeppelin AccessManager correctly enforces `restricted` role bindings
- Robinhood Chain behaves as an EVM (cancun) chain; on stable builds ArbSys(0x64).arbBlockNumber() is the clock when USE_ARB_SYS

ADMIN SEMI-TRUSTED

- Can configure pools and fee bands, set sinks and the claim fee, grant roles
- Cannot move LP principal, set a fee above 10%, or un-configure a pool
- Can change role-to-function bindings in the AccessManager

FEE_POKER SEMI-TRUSTED

- Moves each side's fee within [max(floor, 50% of base), min(pool cap, category cap)] for at most 72 h
- Raises by at most one step per 60 s
- Turns the continuation fee on or off
- Cannot leave those bounds

PAUSER SEMI-TRUSTED

- Pauses deposits, claims, stakes, transfers and revenue sweeps for up to 7 days per call
- Clears pokes; switches off the continuation fee; revokes discounts
- Cannot pause `withdraw` or `unstake`

DISCOUNT_ADMIN SEMI-TRUSTED

- Lists senders (routers) for a discount that never raises a fee or goes below the floor
- Cannot raise any fee

REVENUE_OPS SEMI-TRUSTED

- Sets the claim fee, at most 20% of LP fees
- Cannot touch principal

SEEDER SEMI-TRUSTED

- Initializes a pool the admin has configured (FablesHook builds), at the price it passes
- Cannot configure pools or change existing ones

STABLE_KEEPER SEMI-TRUSTED

- On the stable builds: moves a pool's reference by at most 0.1% per minute
- Sets a pool's minimum fee (at most 65,535 pips)
- Cannot move funds or change any other setting

Token issuers (pool currencies) SEMI-TRUSTED

- May pause their token; while paused, exits that move a non-zero amount of it wait until it is unpaused
- Tokens must not be fee-on-transfer or rebasing, and supply must not exceed 2^127 units

Users (LPs / receipt holders using the ledger) UNTRUSTED

- Call deposit (payable on native pools; refund of unspent msg.value via call), withdraw (never pausable), withdrawAndClaim, claimFees (with maxFeeBps bound), stake (with minGaugeWeight bound; requires ballotGate reporting gauge earning), unstake (never pausable), transfer/transferFrom of ERC-6909 receipts (only unstaked balance)
- Call sweepDistributor and sweepClaimFee permissionlessly (paused hook-wide blocks them)
- Choose payout `to` addresses (refused: address(0), the hook, the PoolManager)
- Can force a range sync (deposit/transfer/stake) which prices the claim-fee backlog at the lowest rate since the range's last sync

Swappers / routers (PoolManager.swap callers) UNTRUSTED

- Swap through any pool; the hook's beforeSwap fallback (FablesFeeFull / FablesStableHookLite) overrides the LP fee each swap
- Whitelisted routers (discount entries) are allow-listed callers, not trusted: discount keyed on the PoolManager caller address
- Swaps move the pool tick, which feeds the CF anchor/surcharge (FeeRef.cfStep) and the stable fee's cached block price

Direct liquidity providers (non-Exclusive builds) UNTRUSTED

- Add/remove liquidity directly through the PoolManager into the same pools as the ledger, in the same or overlapping tick ranges
- Donate to pools (donate is never gated), which raises fee growth of ledger ranges

Voters (ballot-gate-admitted callers of syncRanges) UNTRUSTED

- Call FablesLedgerV2.syncRanges for staked ranges of pools the gate admits them for; forces fee collection and claim-fee index roll for those ranges; never pausable

2. Scope

REPOSITORY	FablesFi/fables
COMMIT	0f59859
BRANCH	main
SCAN TYPE	Full Scan

Scanned **54** of **54** units.

UNIT	FILE
`PoolWordLib`	`src/libraries/PoolWordLib.sol`
`FablesHookCore`	`src/base/FablesHookCore.sol`
`FablesExclusiveLiquidityV2`	`src/base/FablesExclusiveLiquidityV2.sol`
`FablesNativeETHV2`	`src/base/FablesNativeETHV2.sol`
`DiscountLib`	`src/libraries/DiscountLib.sol`
`FablesHookETHExclusive`	`src/FablesHookETHExclusive.sol`

`StableFeeCalculation`
WHOLE CONTRACT + 1 FUNCTION
`calculatePriceRatioX96`
	`src/libraries/StableFeeCalculation.sol`

`FeeRef`	`src/libraries/FeeRef.sol`
`FablesHookExclusive`	`src/FablesHookExclusive.sol`
`FablesStableHookLiteExclusive`	`src/FablesStableHookLiteExclusive.sol`
`FablesHookETH`	`src/FablesHookETH.sol`
`FablesHook`	`src/FablesHook.sol`

`FablesFeeFull`
WHOLE CONTRACT + 1 FUNCTION
`.fallback`
	`src/base/FablesFeeFull.sol`

`FablesStableHookLiteTyped`
WHOLE CONTRACT + 1 FUNCTION
`beforeSwap`
	`src/FablesStableHookLite.sol`

`FablesStableHookLite`
WHOLE CONTRACT + 3 FUNCTIONS
`setMinimumFee`
`updateReferencePrice`
`.fallback`
	`src/FablesStableHookLite.sol`

`FablesFeeEngine`
WHOLE CONTRACT + 6 FUNCTIONS
`setDiscountFilter`
`configurePool`
	`src/base/FablesFeeEngine.sol`

UNIT	FILE

`configureCF`
`setDiscount`
`setCFActive`
`pokeFee`	

`LedgerLibV2`
WHOLE CONTRACT + 5 FUNCTIONS
`sweepDistributor`
`sweepClaimFee`
`claimFeeFor`
`settleUser`
`creditFees`
	`src/libraries/LedgerLibV2.sol`

`FablesStableFee`
WHOLE CONTRACT + 6 FUNCTIONS
`setDiscountFilter`
`setDiscount`
`_poolSqrtPrice`
`updateFeeConfig`
`configureStablePool`
`_getFee`
	`src/base/FablesStableFee.sol`

`FablesLedgerV2`
WHOLE CONTRACT + 12 FUNCTIONS
`setBallotGate`
`transferFrom`
`stake`
`transfer`
`withdrawAndClaim`
`setClaimFee`
`unstake`
`unlockCallback`
`claimFees`
`withdraw`
`_syncRange`
`deposit`
	`src/base/FablesLedgerV2.sol`

3. Ingested Documentation

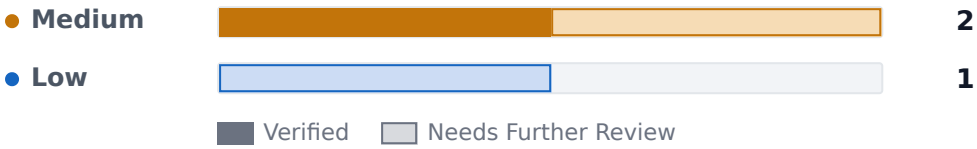
Documentation BugPoCer ingested and used during analysis.

1 DOCUMENT

DOCUMENT	SOURCE	SIZE
`README.md`	Repository Docs	10,223 chars

4. Summary

Scan Started: 29 October 2026 13:31 UTC
Scan Completed: 29 October 2026 15:01 UTC
Report Generated: 29 October 2026 18:58 UTC



2 distinct issues across 3 findings.

ISSUE	SEVERITY	SITES	ASSESSMENT
Fee-window clamp distorts stable decaying fee	Medium	2	Verified + Needs Further Review
Manipulable Spot Anchor	Medium	1	Needs Further Review

SEVERITY

- **High**: Vulnerabilities that enable direct theft or loss of user funds, protocol insolvency, or system-wide failures with realistic attack paths requiring minimal constraints.
- **Medium**: Vulnerabilities that either cause significant impact (fund loss, core functionality breaks) under constrained conditions, moderate impact (temporary DoS, localized losses) under realistic conditions, or minor impact with highly accessible exploit paths.
- **Low**: Vulnerabilities with minimal financial risk that affect non-core functionality, state handling, or minor logic under most conditions, or medium-impact issues requiring admin actions or extreme constraints to exploit.

BADGES ON EACH FINDING

- **Verified**: BugPoCer wrote a proof-of-concept that runs and demonstrates real harm.
- **Needs Further Review**: BugPoCer could not prove it either way; worth a manual review.

5. Findings

5.1. Medium

5.1.1. Fee-window clamp distorts stable decaying fee

MEDIUM 2 FINDINGS

ROOT CAUSE

_getFee overwrites the AMM price with the 10% window edge before _calculateDecayingFee runs, so both the movement adjustment in _decayStartFee and the decay target are computed from the edge rather than the real price.

THE FIX

Pass the unclamped price ratio into _decayStartFee and the target/boundary fee computations, clamping only where a division needs a bounded ratio, and cap the result at STABLE_MAX_FEE.

FINDINGS

a · b

a Fee Window Clamp Suppresses Movement Adjustment

MEDIUM VERIFIED

> **Unit Name:** `beforeSwap`
>
> **Surfaced From:** 3 units
>
> **Location:** src/base/FablesStableFee.sol:_getFee / _decayStartFee (reached via FablesStableHookLite.fallback and FablesStableHookLiteTyped.beforeSwap → _stableFee)
>
> **Touchpoints:** `FablesStableHookLite.fallback` (entry) → `FablesStableHookLiteTyped.beforeSwap` (entry) → `FablesStableFee._stableFee` (intermediate) → `FablesStableFee._decayStartFee` (sink)

DESCRIPTION

Summary: When a stable pool's price jumps past the 10% fee window, the corrective (back-toward-reference) fee is adjusted only up to the window edge. It starts far below min(StablePair fee, 10%), so arbitrageurs keep value that should go to LPs.

Mechanism: `_getFee` replaces any price whose ratio is below `MIN_FEE_PRICE_RATIO_X96` with the edge price before calling `_calculateDecayingFee`. `_decayStartFee` then works out the movement adjustment (`adjustPreviousFeeForPriceMovement`) from edge/prev rather than actual/prev. If the previous price was also clamped, the two prices are equal and the fee is not

adjusted at all. The code comment justifies the clamp by saying the far-boundary fee is already above the 10% cap there. That only holds for fresh far-boundary starts, not for a previous fee that has already decayed and is carried forward.

Attack path:

1. Pool price sits out of band but inside the window (e.g. -9%) and the stored decaying fee has decayed toward its target.
2. In block N a large away-direction swap (0 fee, or the minimum fee) pushes the price past the window to about -20%.
3. In block N+1 `_getFee` clamps -20% to the -10% edge, and `_decayStartFee` adjusts the old fee only for the -9% to -10% move.
4. A backrunning arbitrageur buys back toward the reference at about 1.1% instead of the about 10% StablePair would charge, keeping the difference.

Impact: Example: the pool sits at -9% with a decayed fee of about 0.02%, then a dump moves it to -20% at 0 fee. The next block's corrective swaps pay about 1.1% instead of the 10% cap that StablePair would charge, so LPs lose most of the arbitrage value during depeg moves.

Fix: Calculate the movement adjustment in `_decayStartFee` from the actual (unclamped) current and previous prices. Clamp only the resulting fee (to the edge far-boundary fee or the cap), not the price fed into the adjustment.

Note: PoC shows an untrusted arbitrageur paying 0.78% instead of the capped StablePair fee of 8.47% after a price jump past the 10% window. This costs LPs fees.

EVIDENCE

- `src/base/FablesStableFee.sol:_getFee()` — when `priceRatioX96 < MIN_FEE_PRICE_RATIO_X96`, `sqrtAmmPriceX96` is replaced by the edge price before `_calculateDecayingFee` runs, and that clamped price is also the one stored in `stateLow`.
- `src/base/FablesStableFee.sol:_decayStartFee()` — the movement adjustment uses `calculatePriceRatioX96(sqrtAmmPriceX96, previousSqrtAmmPriceX96)` with the clamped current price; if both prices are clamped they are equal and the branch keeps `previousDecayingFeeE12` unadjusted.
- `src/base/FablesStableFee.sol` constant comment on `MIN_FEE_PRICE_RATIO_X96` — "beyond it the far-boundary fee already exceeds the 10% cap": true for far-boundary starts, false for a previous decayed fee that is carried forward.
- `test/unit/stable/StableLiteParkedPrice.t.sol:testFuzz_beyondTheWindowReadsAsTheEdge` — only covers walks that start inside the band (fresh far-boundary start), so the case of a decayed state followed by a jump past the edge is not tested.
- Observed [2dff1073]: "A deviation from StablePair math, the clamp, the cap or the minimum fee misprices swaps. It can allow cheap arbitrage of LPs."
- src/base/FablesStableFee.sol:392-449 (FablesStableFee._getFee)
- `src/base/FablesStableFee.sol:_getFee()` — when the ratio is below `MIN_FEE_PRICE_RATIO_X96`, `sqrtAmmPriceX96` is overwritten with the edge, passed to `_calculateDecayingFee` and stored as the cached price in `stateLow`.
- `src/base/FablesStableFee.sol:_decayStartFee()` — with previous and current both clamped to the edge, `sqrtAmmPriceX96 < previousSqrtAmmPriceX96` is false, so the upward `adjustPreviousFeeForPriceMovement` branch is skipped and the decayed `previousDecayingFeeE12` is kept.
- `src/base/FablesStableFee.sol:_calculateDecayingFee()` — `targetFeeE12` comes from the boundary fees at the clamped 0.9 ratio, so the fee can decay below 10% even where StablePair's raw-ratio target stays above the cap.
- `src/base/FablesStableFee.sol` constants comment — "beyond it the far-boundary fee already exceeds the 10% cap" justifies the clamp from the starting fee only and ignores the decay and the movement adjustment.
- README — "a price more than 10% from the reference is treated as exactly 10% away", presented as a change that only bounds the fee, not one that removes StablePair's protection on the swap-back fee.

PROOF OF CONCEPT

`test/poc/StableClampSuppressesMovementAdjustment_e2f19312.t.sol` — Deploys the reference `StablePair` and `FablesStableHookLite` side by side with identical config and liquidity. It moves the price to -9%, lets the fee decay, then dumps the price to -20% at 0 fee. In the next block, a corrective swap on the Lite pool is charged a much lower fee than the capped `StablePair` fee (`min(StablePair, 10%)`), because `_getFee` clamps the price

before `_decayStartFee` computes the movement adjustment. The assertions on the fee mismatch and the arbitrageur's extra token0 output fail, which reproduces the issue, so this is a true positive.

```
// SPDX-License-Identifier: BUSL-1.1
pragma solidity ^0.8.26;

import "test/utils/StableTestBase.sol";

/// @notice PoC: when a stable pool's price jumps past the 10% fee window,
FablesStableFee._getFee clamps the price to
///         the window edge BEFORE _decayStartFee computes the movement
adjustment. The carried-forward (already
///         decayed) fee is therefore only adjusted for the move up to the edge,
so the corrective (back toward the
///         reference) fee is far below min(StablePair fee, 10%), the design
target the repo's own tests assert.
///
///         Side-by-side: the deployed Uniswap StablePair bytecode and
FablesStableHookLite, identical config,
///         identical liquidity, identical swap sequence.
contract StableClampSuppressesMovementPoC is StableTestBase {
    using PoolIdLibrary for PoolKey;
    using StateLibrary for IPoolManager;

    SeqSwapper internal sw;
    address internal sp;
    address internal lite;
    PoolKey internal ksp;
    PoolKey internal kl;

    struct Out {
        uint24 fee;
        int128 a0;
        int128 a1;
    }

    function setUp() public {
        setUpChain();
        sw = new SeqSwapper(pm);
        MockERC20(Currency.unwrap(c0)).mint(address(sw), BIG);
        MockERC20(Currency.unwrap(c1)).mint(address(sw), BIG);

        sp = _deploySP("clamp-sp");
        lite = _deployLite("clamp-lite");

        // Default-looking stable config: k = 0.7 per block, 0.005% band,
targetMultiplier 100, reference = _refSqrt()
        StableFeeConfig memory c =
            StableFeeConfig({k: K07, optimalFeeE6: 50, targetMultiplier: 100,
referenceSqrtPriceX96: _refSqrt()});
        ksp = PoolKey(c0, c1, DYN, 1, IHooks(sp));
        kl = PoolKey(c0, c1, DYN, 1, IHooks(lite));
        _createPool(sp, ksp, _refSqrt(), c);
        _createPool(lite, kl, _refSqrt(), c);

        // identical liquidity on both pools, covering the reference down to
about -26%
        _addLiq(sp, ksp, -887272, 887272, 1e21);
```

```
        _addLiq(lite, kl, -887272, 887272, 1e21);
        _addLiq(sp, ksp, INIT_TICK - 3000, INIT_TICK + 600, 1e24);
        _addLiq(lite, kl, INIT_TICK - 3000, INIT_TICK + 600, 1e24);
    }

    function _swapOne(PoolKey memory k, bool zfo, int256 amt, uint160 lim)
internal returns (Out memory o) {
        vm.recordLogs();
        (bool ok,) = sw.swap(k, zfo, amt, lim);
        require(ok, "swap failed");
        Vm.Log[] memory logs = vm.getRecordedLogs();
        bool found;
        for (uint256 j; j < logs.length; j++) {
            if (
                logs[j].emitter == PM_ADDR && logs[j].topics[0] == SWAP_TOPIC
                    && logs[j].topics[1] == PoolId.unwrap(k.toId())
            ) {
                (o.a0, o.a1,,,, o.fee) = abi.decode(logs[j].data, (int128,
int128, uint160, uint128, int24, uint24));
                found = true;
            }
        }
        require(found, "no Swap event");
    }

    function _walkBoth(uint160 to) internal returns (uint24 fsp, uint24 fl) {
        (uint160 p,,,) = pm.getSlot0(ksp.toId());
        bool zfo = to < p;
        fsp = _swapOne(ksp, zfo, -1e30, to).fee;
        fl = _swapOne(kl, zfo, -1e30, to).fee;
        (uint160 a,,,) = pm.getSlot0(ksp.toId());
        (uint160 b,,,) = pm.getSlot0(kl.toId());
        require(a == to && b == to, "walk did not reach target");
    }

    function test_priceJumpPastWindow_correctiveFeeFarBelowCappedStablePair()
public {
        uint256 ref = _refSqrt();
        // price ratio 0.91 (-9%): out of band, inside the 10% window
        uint160 p9 = uint160(ref * 953_939 / 1_000_000);
        // price ratio 0.80 (-20%): beyond the 10% window
        uint160 p20 = uint160(ref * 894_427 / 1_000_000);

        // Block B0: the price drifts to -9% (first swap of the block reads the
in-band price).
        _rollClock(1);
        _walkBoth(p9);

        // Many blocks pass: the out-of-band decaying fee will have decayed
toward its target.
        _rollClock(100);

        // Block B1: a large away-direction dump pushes the price past the
window to -20%. Its first-of-block
        // beforeSwap stores state at the -9% price with the decayed fee
(identical on both hooks).
```

```
        (uint24 dumpSp, uint24 dumpL) = _walkBoth(p20);
        assertEq(dumpSp, 0, "away swap pays 0 on StablePair");
        assertEq(dumpL, 0, "away swap pays 0 on Lite");
        (uint40 dSp, uint160 sSp,) = IStablePair(sp).feeState(ksp.toId());
        (uint40 dL, uint160 sL,) = IStablePair(lite).feeState(kl.toId());
        assertEq(uint256(dSp), uint256(dL), "pre-jump decayed fee identical");
        assertEq(uint256(sSp), uint256(sL), "pre-jump cached price identical");
        console.log("decayed fee carried forward (E12):", uint256(dL));

        // Block B2: the arbitrageur backruns toward the reference (1for0 is
corrective below the reference).
        _rollClock(1);
        (, uint24 qSp) = _getFee(sp, ksp);
        (, uint24 qL) = _getFee(lite, kl);
        console.log("StablePair quoted corrective fee (pips):", uint256(qSp));
        console.log("Lite quoted corrective fee (pips):     ", uint256(qL));
        console.log("min(StablePair, 10%) (pips):           ",
uint256(_capped(qSp)));

        // Same exact-input trade back on both pools.
        Out memory oSp = _swapOne(ksp, false, -1e21, TickMath.MAX_SQRT_PRICE -
1);
        Out memory oL = _swapOne(kl, false, -1e21, TickMath.MAX_SQRT_PRICE - 1);
        console.log("StablePair charged fee (pips):", uint256(oSp.fee));
        console.log("Lite charged fee (pips):      ", uint256(oL.fee));
        console.log("token0 out on StablePair:", uint256(uint128(oSp.a0)));
        console.log("token0 out on Lite:      ", uint256(uint128(oL.a0)));

        // The Lite build's documented target is min(StablePair fee, 10%). Fails
when the clamp suppresses the
        // movement adjustment: Lite charges ~0.8% where the capped StablePair
fee is ~8.5%.
        assertEq(oL.fee, _capped(oSp.fee), "Lite corrective fee !=
min(StablePair fee, 10%): clamp suppressed adjustment");
        // LP harm: the arbitrageur receives more token0 for the same token1
input on the Lite pool.
        assertLe(uint256(uint128(oL.a0)), uint256(uint128(oSp.a0)), "arbitrageur
extracts extra value from Lite LPs");
    }
}
```

b Fee Window Clamp Lowers Decay Target

LOW NEEDS FURTHER REVIEW

Entry Point: `FablesStableHookLite.fallback`

Location: src/FablesStableHookLite.sol: FablesStableHookLite.fallback (beforeSwap) → src/base/FablesStableFee.sol: _stableFee → _getFee → _calculateDecayingFee (targetFeeE12)

Touchpoints: `FablesStableHookLite.fallback` (entry) → `FablesStableFee._stableFee` (intermediate) → `FablesStableFee._getFee` (intermediate) → `FablesStableFee._calculateDecayingFee` (sink)

DESCRIPTION

Summary: When a stable pool's price is more than 10% from the reference, the falling fee on swaps back toward it decays toward a target computed at the window edge, not at the real price. Arbitrageurs pay well below min(StablePair fee, 10%).

Mechanism: `_getFee` moves `sqrtAmmPriceX96` / `priceRatioX96` to the window edge, then computes `closeBoundaryFeeE12` and `calculateFarBoundaryFee` from that edge ratio. `_calculateDecayingFee` sets `targetFeeE12 = far - close * targetMultiplier / 100` from those edge values. The comment justifies the clamp by saying the far-boundary fee there is already above the 10% cap. That holds for the far fee but not the target, which grows with distance for targetMultiplier < 100. So the falling fee settles below the capped StablePair value.

Impact: Example: optimalFeeE6 = 100, targetMultiplier = 50, price parked at 0.8×reference. StablePair's target is about 10.0%, so the capped fee stays at 10%. Fables decays to about 5.0%, so swaps back toward the reference pay about half the LP fee while the price stays outside the window.

Fix: Compute `targetFeeE12` (close and far boundary fees) from the unclamped price ratio and keep the clamp only where a division by the ratio needs it. Alternatively, raise the target to at least min(the actual-price target, STABLE_MAX_FEE) before decaying.

Note: Assessed as low impact; reported without a proof of concept.

EVIDENCE

- `src/base/FablesStableFee.sol:_getFee()` — when `priceRatioX96 < MIN_FEE_PRICE_RATIO_X96`, it overwrites `sqrtAmmPriceX96` and `priceRatioX96` with the edge values, then computes `closeBoundaryFeeE12` and the far-boundary fee from the edge ratio and passes them to `_calculateDecayingFee`.
- `src/base/FablesStableFee.sol:_calculateDecayingFee()` — `targetFeeE12 = farBoundaryFeeE12 - closeBoundaryFeeE12 * tm / 100`, built from edge values, is where `calculateDecayingFee` settles.
- `src/base/FablesStableFee.sol` constants comment — says "beyond it the far-boundary fee already exceeds the 10% cap", which assumes the clamp never lowers the fee charged; it does lower the decay target.
- `src/base/FablesStableFee.sol:_decayStartFee()` — when the price stays parked, the previous and current clamped prices are equal, so the start is the previous decayed fee and nothing pulls the fee back above the edge target.
- Distinct from e2f19312: fixing the movement adjustment in `_decayStartFee` leaves `targetFeeE12` computed at the edge.

5.1.2. Manipulable Spot Anchor

MEDIUM NEEDS FURTHER REVIEW

> **Unit Name:** `setCFActive`
>
> **Location:** src/base/FablesFeeEngine.sol:setCFActive

DESCRIPTION

Summary: `setCFActive(true)` sets the CF anchor to whatever the slot0 tick is when the call runs. Whoever lands the FEE_POKER's call can move the price first and move it back after, leaving the anchor at a tick they picked. Swappers on one side then pay up to cfCap extra.

Mechanism: The code reads `poolManager.getSlot0(poolId)` and calls `FeeRef.cfRearm(cf, block.timestamp, tick)`. Nothing checks that the PoolManager is locked and the caller gives no expected tick, so a mid-unlock tick is accepted. The call also works while the CF is already on. Because `tLast = now` and `period >= 1`, the attacker's swap back in the same transaction does not update the anchor (`dt < period` in `cfStep`). The `CFActive` revert added to `configureCF` blocks this exact relayed-at-a-moved-price attack only there. Its own comment hands the job to `setCFActive`, which has no such guard.

Attack path:

1. Attacker sees a fully signed FEE_POKER Safe tx calling `setCFActive(id, true)`, e.g. on the public Safe transaction service.
2. Inside `PoolManager.unlock`, attacker swaps zeroForOne to push the tick down to Tm.
3. Attacker executes the Safe tx from the callback; `cfRearm` stores anchor = Tm*256 and tLast = now.
4. Attacker swaps back to T0; `dt=0 < period`, so the anchor stays at Tm and the swap back pays no surcharge.
5. For up to 255 s, oneForZero swappers pay `min(k*(T0-Tm)256100>>16, cfCap)` extra pips, which go to LPs.

Impact: After a relayed or sandwiched switch-on, swaps in one direction pay up to cfCap (25 bp) above their fee for up to maxAge (255 s). An attacker holding liquidity in the pool captures this. It needs a pending FEE_POKER switch-on that others can execute or front-run.

Fix: In `setCFActive`, revert when the PoolManager is unlocked (`TransientStateLibrary.isUnlocked`). Also take an expected tick and a maximum deviation from the caller, so a sandwiched tick is refused.

EVIDENCE

- `src/base/FablesFeeEngine.sol:setCFActive()` — anchors with `FeeRef.cfRearm(cf, block.timestamp, tick)` using the live `getSlot0` tick. It has no unlocked-state check, no expected-tick bound, and no check that the CF is currently off.
- `src/base/FablesFeeEngine.sol:configureCF()` — reverts `CFActive` while on 'so the anchor cannot be picked by moving the price first', and defers anchoring to `setCFActive(true)`, which is open to the same attack.
- `src/libraries/FeeRef.sol:cfStep()` — the anchor only updates when `dt >= period`. With tLast just set to now, a swap back in the same transaction leaves the moved anchor in place.
- `test/regression/ConfigureCFWhileOn.t.sol:MovingRelayer` — the project's own threat model: an admin call relayed inside an unlock after a price move. Only `configureCF` is covered.
- `src/libraries/FeeRef.sol:resolve()` — when `cfOn`, adds the surcharge (up to cfCap) to every swap in the continuing direction.